

Analysis and Comparison of EKF-SLAM and FastSLAM 1.0 with Known Data Association in Indoor Environments

Lorenzo Manno, lmanno@kth.se
KTH Royal Institute of Technology

January 15, 2026

Abstract

The growing interest in Simultaneous Localization and Mapping (SLAM) is driven by the increasing demand for autonomous systems, particularly in mobile robotics and autonomous driving applications. SLAM represents a fundamental component of modern robotic perception pipelines, enabling a robot to concurrently estimate its own pose while building a map of an environment.

This paper presents an implementation and comparative analysis of two widely adopted online SLAM approaches: EKF-SLAM and FastSLAM. Both methods are evaluated on the UTIAS Multi-Robot Cooperative Localization and Mapping dataset for the known data association case. The comparison focuses on estimating the errors on robot's Pose and map construction over time, highlighting the strengths and limitations of each approach in a real scenario.

1 Introduction

Simultaneous Localization and Mapping (SLAM) refers to a class of algorithms that address two tightly coupled problems simultaneously. In the context of mobile robotics, SLAM enables a robot to estimate its own position in space while concurrently building a map of the surrounding environment. These algorithms represent a fundamental component in the development of autonomous systems, particularly in applications such as autonomous driving. Furthermore, the field of autonomous systems is experiencing rapid growth, driven by significant investments from major industry leaders. Its applications span diverse sectors, including space exploration, defense, industrial automation, and healthcare. Consequently, the study of enabling technologies and algorithms, such as SLAM, is of primary importance.

In this work, we focus on the implementation and analysis of two widely adopted SLAM approaches. The first is EKF-SLAM, based on the Extended Kalman Filter.

While EKF-SLAM benefits from relatively low implementation complexity, it is subject to strict assumptions that limit its applicability in certain scenarios. The second approach is FastSLAM, which relies on Monte Carlo methods and particle filtering. This method typically adapts better to many real-world situations but often faces challenges regarding execution time when a large number of particles is required.

The objective of this project is to evaluate and compare the performance of these algorithms for online SLAM in mobile robotics. They are evaluated based on the accuracy of the robot's pose estimation and the precision of the generated environmental map using various metrics. Through this analysis, we aim to highlight the strengths and weaknesses of each method to determine which is best suited for specific scenarios. This work utilizes the UTIAS (MR.CLAM) dataset, which features a small indoor environment with known measurement-landmark associations. This setting could resemble an automated warehouse, where autonomous robots are extensively used in industry. Consequently, the presented results illustrate the inherent trade-offs, facilitating an informed choice based on the constraints of different applications.

1.1 Outline

The remainder of this paper is structured as follows:

1. **Section 2** reviews related work in the field of SLAM, discussing the main features constituting EKF and Fast SLAM and their primary applications, as well as the benchmark methodologies used to evaluate their performances. Lastly, it also includes the specifics about the dataset used for this work.
2. **Section 3** describes the methodology, including the motion and measurement models used for mobile robots, the theoretical framework of EKF-SLAM and FastSLAM, accompanied by their respective pseudocodes and explanations. Finally we introduce the metrics used to evaluate those algorithms.
3. **Section 4** details the implementation specifics, including some important choices made for the evaluation, like the extraction of pose and map from particles, and additions made to the standard algorithms.

4. **Section 5** presents the experimental setup, and the obtained results along with a discussion of their implications in real applications.
5. **Section 6** concludes the paper by summarizing our scope and results and outlining possible directions for future work.

2 Related Work

The problem of Simultaneous Localization and Mapping (SLAM) has been extensively studied over the past decades, evolving from theoretical estimation problems to robust, real-time systems. In this section we introduce the two SLAM algorithms, as well as some of their primary characteristics. Secondly, we introduce the works that led to our benchmarking strategy and lastly the dataset used for evaluation.

2.1 EKF-SLAM

The theoretical foundations of the stochastic mapping problem were established in the seminal work by Smith and Cheeseman [8]. They formulated SLAM as an estimation problem using the Extended Kalman Filter (EKF) to jointly estimate robot pose and landmark locations, specifically addressing the case of feature-based maps with known data association. Following this theoretical breakthrough, one of the first practical implementations of EKF-SLAM was demonstrated by Leonard and Durrant-Whyte [3], who validated the approach on a mobile robot using laser range data.

While these early works defined the core mechanics of the filter, the specific algorithmic implementation adopted in this paper follows the standardized formulation presented by Thrun, Burgard, and Fox in "Probabilistic Robotics" [10], which serves as the primary reference for this work.

This approach relies on the fundamental assumption that measurement and motion noise are governed by Gaussian distributions and that non-linear system dynamics can be locally linearized via first-order Taylor expansion. Consequently, EKF-SLAM operates optimally in environments where non-linearities are moderate and the robot's uncertainty remains bounded. However, these assumptions introduce significant limitations. First, the linearization error can accumulate over time, leading to filter divergence if the initial uncertainty is too high. Second, the algorithm maintains a single mean and covariance for the entire state vector, meaning it cannot represent multi-modal distributions. Finally, the computational cost of the measurement update step grows quadratically with the number of landmarks ($O(K^2)$), rendering the standard EKF computationally prohibitive for mapping large-scale environments.

2.2 Particle Filter Approaches

To address the limitations inherent in EKF-SLAM, as well as its quadratic computational complexity ($O(K^2)$)

with respect to the number of landmarks, Montemerlo et al. introduced FastSLAM [5]. This approach utilizes a Rao-Blackwellized Particle Filter to decompose the full SLAM problem into a robot localization component, estimated via a set of M particles, and a conditional mapping component, where each landmark is estimated by an independent EKF.

A key advantage of this factorization is its potential for efficient scaling in large-scale environments. While a naive implementation scales linearly as $O(M \cdot K)$, FastSLAM can theoretically achieve a computational complexity of $O(M \log K)$ by utilizing balanced binary trees to manage the map data structures.

Literature typically distinguishes between two variants of this algorithm based on their proposal distribution. FastSLAM 1.0 samples poses directly from the probabilistic motion model. While efficient, this approach ignores the current measurement during the proposal step. Consequently, when the motion noise is high relative to the measurement noise, particles may be distributed in low-likelihood regions, leading to rapid particle depletion. To mitigate this, FastSLAM 2.0 was proposed [6], which incorporates the current measurement into the proposal distribution. By sampling from the posterior rather than the motion model, FastSLAM 2.0 places particles in regions of high likelihood, significantly improving filter convergence and reducing the number of required particles.

2.3 Evaluation Metrics

A seminal contribution to SLAM evaluation is the work by Sturm et al. [9]. This work established community standards for quantifying trajectory consistency and global accuracy using metrics that rely on precise ground truth data.

Absolute Trajectory Error (ATE): This metric is primarily used to assess the global consistency of the estimated trajectory by comparing the absolute distance between the estimated pose and the ground truth. Since the SLAM system and the ground truth often operate in different coordinate frames, the estimated trajectory must first be aligned. This is typically achieved using the closed-form solution for absolute orientation derived by Horn [2], which minimizes the least-squares error between the two point clouds via Singular Value Decomposition.

Relative Pose Error (RPE): Unlike ATE, which measures global drift, RPE measures the local accuracy of the trajectory over a fixed time interval. This metric is instrumental in measuring the drift of the system per unit of distance or time. Furthermore, by analyzing the relative error over larger intervals, RPE can be used to detect and evaluate loop closures, as successful corrections typically result in a significant reduction of relative error between revisited locations.

2.4 Dataset

To validate the implemented SLAM algorithms, we selected the UTIAS Multi-Robot Cooperative Localization and Mapping (MR.CLAM) dataset [4]. We chose this

dataset to ensure the algorithms were tested on real data reflecting authentic operating scenarios, rather than relying solely on synthetic simulations.

The physical environment consists of a 15m×8m indoor workspace populated with 15 static cylindrical landmarks and a fleet of five differential-drive robots.

The sensory data includes both proprioceptive and exteroceptive measurements. For odometry, the robots log linear (v) and angular (ω) velocity commands at a frequency of 67 Hz. Exteroceptive sensing is achieved via monocular cameras mounted on each robot; these sensors detect unique visual barcodes placed on the landmarks and robots, providing identification numbers alongside relative range and bearing measurements at approximately 10 Hz. Since in our experiments we are going to use only one robot at a time, the presence of barcodes makes it trivial to filter among the measurements. Furthermore, the dataset offers millimeter-accurate ground truth from a Vicon motion capture system logging robot’s position at a frequency of 100Hz, enabling a precise quantitative evaluation of the metrics presented in the previous section.

3 Methodology

This section presents the models and algorithms used in this work. Both EKF-SLAM and FastSLAM 1.0 were implemented assuming known correspondences between measurements and landmarks. This assumption is justified by the characteristics of the UTIAS MR.CLAM dataset [4], where each landmark is equipped with a unique visual barcode. The perception system directly provides the landmark identifier together with the corresponding measurement, eliminating ambiguity in data association.

3.1 Mobile Robot Models

The motion and observation models used in this work are dictated by the sensing modalities and robot kinematics provided in the UTIAS dataset. All robots are differential-drive platforms, and exteroceptive measurements consist of range and bearing observations to known landmarks. The robot motion is described by a discrete-time differential-drive model. Given the linear velocity v and angular velocity ω over a time interval Δt , the robot pose evolves according to:

$$\mathbf{x}_{t+1} = \begin{bmatrix} x_t + v \cos(\theta_t) \Delta t \\ y_t + v \sin(\theta_t) \Delta t \\ \theta_t + \omega \Delta t \end{bmatrix}, \quad (1)$$

where $\mathbf{x}_t = [x_t, y_t, \theta_t]^T$ represents the robot pose at time t .

This model directly reflects the odometry information provided in the dataset, which consists of time-stamped linear and angular velocity commands.

Regarding the measurement model used in the update steps, each landmark is modeled as a static point in the plane. The measurement model is defined in terms of

range r and bearing b , which is consistent with how observations are provided in the dataset. For landmark i at position $\mathbf{m}_i = [m_{i,x}, m_{i,y}]^T$, the observation model is:

$$\mathbf{z}_t^i = \begin{bmatrix} \sqrt{(m_{i,x} - x_t)^2 + (m_{i,y} - y_t)^2} \\ \text{atan2}(m_{i,y} - y_t, m_{i,x} - x_t) - \theta_t \end{bmatrix}. \quad (2)$$

Both motion and measurement noise are assumed to be zero-mean Gaussian and are modeled through their respective covariance matrices.

3.2 SLAM Algorithms

This subsection describes the two SLAM algorithms implemented in this work: EKF-SLAM and FastSLAM 1.0. Both implementations follow the formulations presented in [10].

3.2.1 EKF-SLAM

EKF-SLAM represents the full SLAM posterior using a single multivariate Gaussian. The state vector includes the robot pose and all landmark positions:

$$\mu = [x, y, \theta, m_{1,x}, m_{1,y}, \dots, m_{K,x}, m_{K,y}]^T. \quad (3)$$

Algorithm 1 summarizes the EKF-SLAM procedure. Lines 2–4 perform the prediction step, where the robot pose is propagated using the motion model and the covariance is updated via linearization. Lines 6–18 correspond to the update step, which incorporates landmark observations. When a landmark is observed for the first time (lines 8–10), its position is initialized using the inverse measurement model. For previously observed landmarks, the expected measurement and its Jacobian are computed (lines 12–14), followed by the Kalman gain calculation and state update (lines 15–17).

3.2.2 FastSLAM 1.0

FastSLAM uses a Rao-Blackwellized Particle Filter to factorize the SLAM posterior into a robot trajectory estimation problem and independent landmark estimation problems conditioned on the trajectory. Each particle represents a hypothesis over the robot pose history and maintains an independent map.

Algorithm 2 outlines the FastSLAM 1.0 procedure. Lines 2–4 sample a new robot pose for each particle using the motion model. Lines 6–14 update the landmark estimates using EKFs conditioned on the sampled pose. The importance weight of each particle is computed from the measurement likelihood, and resampling is performed to focus computation on high-probability hypotheses.

In theory, FastSLAM can achieve a computational complexity of $O(M \log K)$ by employing balanced binary trees for landmark management as we presented in the previous section. However, in this work a simpler $O(MK)$ implementation was adopted. This choice is justified by the limited size of the maps in the UTIAS dataset, which contain only 15 landmarks. For such small-scale environments, the overhead introduced by more complex data structures would not result in meaningful performance gains.

Algorithm 1 EKF-SLAM with Known Correspondences

```
1: Initialize  $\mu_0, \Sigma_0$ 
2: for each time step  $t$  do
3:   Predict state using motion model
4:   Compute Jacobian  $G_t$ 
5:    $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + Q_t$ 
6:   for each measurement  $z_t^i$  do
7:     if landmark  $i$  not initialized then
8:       Initialize landmark position
9:     end if
10:    Compute expected measurement  $\hat{z}_t^i$ 
11:    Compute Jacobian  $H_t^i$ 
12:    Compute Kalman gain  $K_t^i$ 
13:    Update mean and covariance
14:   end for
15: end for
```

Algorithm 2 FastSLAM 1.0 with Known Correspondences

```
1: Initialize particle set
2: for each time step  $t$  do
3:   for each particle  $k$  do
4:     Sample pose from motion model
5:     for each measurement  $z_t^i$  do
6:       if landmark  $i$  not initialized then
7:         Initialize landmark
8:       else
9:         Update landmark EKF
10:        Compute particle weight
11:      end if
12:    end for
13:   end for
14:   Resample particles according to weights
15: end for
```

3.3 Evaluation Methodology

To quantitatively assess the performance of the implemented algorithms, we evaluate both the consistency of the estimated map and the accuracy of the robot's trajectory using three standard metrics.

3.3.1 Landmark RMSE

The mapping accuracy is evaluated using the Root Mean Square Error (RMSE) of the estimated landmark positions. Let N be the number of landmarks, $\mathbf{m}_i^{gt}$ be the ground truth position of landmark i , and $\mathbf{m}_i^{est}$ be the estimated position. The metric is defined as:

$$RMSE_{map} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\mathbf{m}_i^{gt} - \mathbf{m}_i^{est}\|^2}. \quad (4)$$

For FastSLAM, $\mathbf{m}_i^{est}$ is the position associated with the particle having the highest weight at the final time step.

3.3.2 Absolute Trajectory Error (ATE)

ATE measures the global consistency of the estimated trajectory by comparing it directly to the ground truth. Since the SLAM estimate and ground truth may be expressed in different reference frames, they must first be aligned. We utilize the method by Horn [2], which computes the optimal rigid transformation $S \in SE(2)$ (rotation and translation) that minimizes the least-squares error between the two trajectories. Given the aligned estimated pose sequence $\hat{\mathbf{x}}_{1:T}$ and the ground truth sequence $\mathbf{x}_{1:T}$, the ATE at time t is the translational difference:

$$ATE_t = \|\text{trans}(\mathbf{x}_t) - \text{trans}(\mathbf{S} \cdot \hat{\mathbf{x}}_t)\|. \quad (5)$$

The global metric is reported as the RMSE of these errors over the entire trajectory:

$$RMSE_{ATE} = \sqrt{\frac{1}{T} \sum_{t=1}^T ATE_t^2}. \quad (6)$$

3.3.3 Relative Pose Error (RPE)

RPE measures the local accuracy (drift) of the trajectory over a fixed time interval Δ . Let P_t and Q_t be the estimated and ground truth pose matrices in $SE(2)$ at time t . The error matrix E_t compares the relative motion of the estimate against the ground truth:

$$E_t = (Q_t^{-1} Q_{t+\Delta})^{-1} (P_t^{-1} P_{t+\Delta}). \quad (7)$$

The metric is reported as the RMSE of the translational component of E_t over all possible time steps $M = T - \Delta$:

$$RMSE_{RPE} = \sqrt{\frac{1}{M} \sum_{t=1}^M \|\text{trans}(\mathbf{E}_t)\|^2}. \quad (8)$$

The reason why we ignore the orientation in our metrics is that an orientation error manifests as a translational

one over time, as described by [9], so we decided to not consider it. In this work, we use $\Delta = 100$ to evaluate drift over around 1.5 seconds intervals.

4 Implementation

Beyond the standard theoretical formulations, several specific engineering design choices were made to enhance numerical stability and estimation accuracy.

4.1 Velocity-Dependent Process Noise

A critical factor in filter performance is the tuning of the process noise covariance Q_t . For EKF-SLAM, we observed that using a static covariance matrix resulted in erratic and "bouncy" trajectories, as can be noticed in Figure 1. To address this, we implemented a velocity-dependent noise model adapted from Thrun et al.[10], where uncertainty scales with the control inputs:

$$\mathbf{Q}_t = \begin{bmatrix} \alpha_1 v_t^2 + \alpha_2 \omega_t^2 & 0 \\ 0 & \alpha_3 v_t^2 + \alpha_4 \omega_t^2 \end{bmatrix}, \quad (9)$$

This formulation prevents the filter from becoming overconfident during aggressive maneuvers. As illustrated in Figure 2, this adaptation significantly smoothed the EKF trajectory and reduced the Absolute Trajectory Error (ATE). In contrast, we found that FastSLAM already produced stable, low-error estimates using only a static covariance. While we conducted experiments using the adaptive model, the results showed no relevant improvement in performance. Therefore, we decided to retain the simpler static noise model for FastSLAM

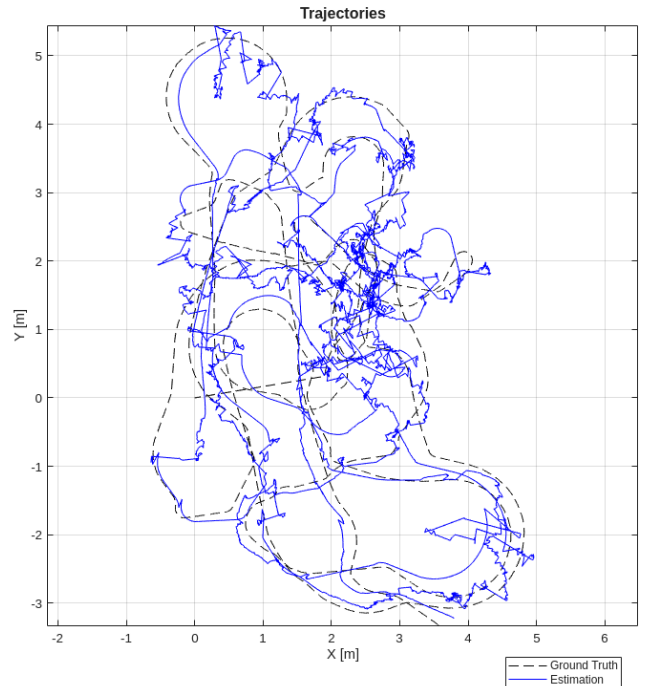


Figure 1: EKF-SLAM Trajectory estimate with static process noise

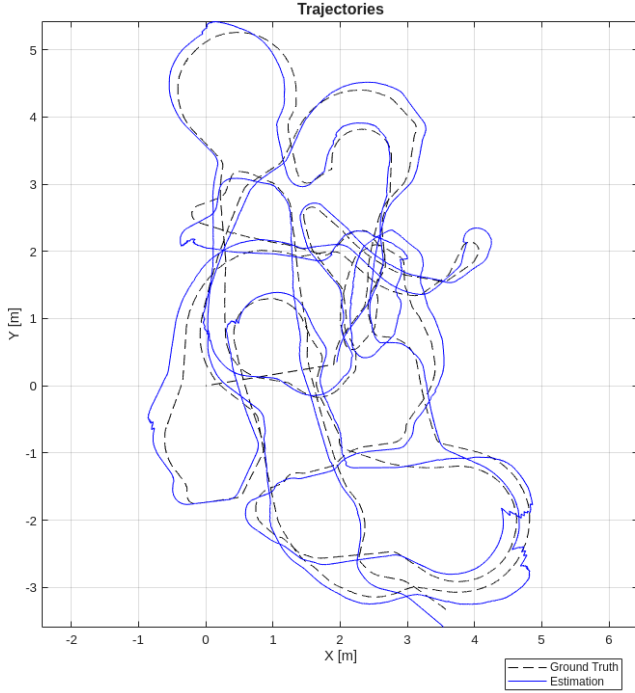


Figure 2: EKF-SLAM Trajectory estimate with adaptive process noise

4.2 FastSLAM Specific Optimizations

The particle filter nature of FastSLAM introduces stochastic challenges not present in the EKF. To address these, we integrated two standard mechanisms to ensure filter consistency.

To enhance the robustness and numerical stability of the FastSLAM algorithm, we implemented two improvements: adaptive resampling and measurement validation via soft gating. To address particle degeneracy without inducing sample impoverishment, we utilized adaptive resampling controlled by the effective sample size (N_{eff}), triggering the resampling step only when N_{eff} drops below 50% of the total particle count, as recommended by Bergman [1] and Thrun et al. [10]. Furthermore, to filter out possible spurious data, we employed a Normalized Innovation Squared (NIS) gate; measurements exceeding the χ^2 significance threshold are considered outliers, leading to a skipped update and a minor weight penalty rather than a hard rejection [10].

4.3 Data Extraction Strategies

Finally, we must define how the internal states of the filters are extracted for comparison against the ground truth.

4.3.1 Trajectory Estimate Extraction

An additional challenge in FastSLAM is condensing the cloud of M particles into a single reference trajectory for evaluation, while in EKF-SLAM we just use the robot's

pose from the state vector at each time step. We explored four techniques: selecting the particle with the highest weight (Best Particle), calculating the weighted mean, K-Means clustering, and the unweighted mean.

Our analysis revealed that the "Best Particle" method and the K-Means produced erratic, discontinuous trajectories due to rapid pose switches between time steps. Conversely, the Unweighted Mean of the particle set (including a circular mean for the heading θ) consistently produced the smoothest and most accurate approximation of the true path. Therefore, all FastSLAM trajectory results reported in this work are derived using the unweighted mean of the particle distribution.

4.3.2 Map Estimate Extraction

For EKF-SLAM, the final map is simply the mean vector of landmark positions extracted from the state vector at the final time step. While for FastSLAM, we take the landmarks estimates of the most weighted particle at the final time step.

We hypothesized that the map at the final time step might not necessarily be the optimal one due to potential drift accumulation. Consequently, we investigated alternative strategies, such as extracting the map at time steps where the aggregate covariance trace was minimal in EKF-SLAM (indicating high confidence, typically after a loop closure). However, preliminary tests showed no significant improvement in average mapping error compared to the simpler approach. Thus, for consistency, all reported map errors are based on the state at the final time step.

5 Experimental Results

In this section, we present the quantitative and qualitative evaluation of the implemented SLAM algorithms. The experiments were conducted on the UTIAS dataset, specifically with Robot 1, on Dataset 1 and Dataset 3.

We compare the EKF-SLAM against FastSLAM 1.0 with varying particle counts $M \in \{50, 300, 500\}$ to analyze the trade-off between computational cost and estimation accuracy.

5.1 Experimental Setup

To strictly evaluate the algorithms under realistic conditions, we implemented a specific processing pipeline to handle the asynchronous nature of the UTIAS dataset. Unlike simulation environments with fixed time steps, the dataset provides odometry (approx. 67 Hz) and visual measurements (approx. 10 Hz) at varying intervals. Consequently, we adopted an asynchronous "sample-and-hold" architecture: the prediction step is triggered dynamically by the arrival of any sensor event, utilizing the exact time elapsed Δt since the previous update. Control inputs (v, ω) are updated only upon the receipt of odometry messages and are held constant during the inter-arrival times.

The UTIAS dataset contains data from five robots simultaneously navigating the same map. For our evaluation, we focused exclusively on **robot 1**, discarding all sensor measurements originating from the other robots. This selection is straightforward, as each robot carries a unique barcode, allowing unambiguous identification of the measurements.

For the quantitative evaluation, precise synchronization between the estimated trajectory and the ground truth is required. Since the Vicon motion capture system provides data at a high frequency of 100 Hz, the temporal discretization error is minimal. To align the asynchronous filter estimates with the ground truth, we employed linear interpolation based on the timestamps. Those techniques have been employed also in other implementations of those algorithms on this dataset, like in [7].

5.2 Results

Tables 1 and 2 summarize the accuracy metrics (ATE, RPE, and Map RMSE) over the entire trajectory for Dataset 1 and Dataset 3, respectively.

Regarding computational efficiency, EKF-SLAM proved to be significantly faster than FastSLAM. On both datasets, the EKF-SLAM algorithm completed the processing of the full trajectory in the order of seconds. In contrast, FastSLAM 1.0 exhibited a substantial increase in computational cost, with an execution time in the order of minutes, scaling linearly with the number of particles used, since the number of landmarks is fixed.

Table 1: Performance Comparison on Dataset 1 (Robot 1)

Algorithm	ATE [m]	RPE _{$\Delta=100$} [m]	Map RMSE [m]
EKF-SLAM	0.179	0.025	0.452
FastSLAM (50 parts)	2.375	0.072	1.081
FastSLAM (300 parts)	0.349	0.055	0.608
FastSLAM (500 parts)	0.158	0.032	0.382

Table 2: Performance Comparison on Dataset 3 (Robot 1)

Algorithm	ATE [m]	RPE _{$\Delta=100$} [m]	Map RMSE [m]
EKF-SLAM	0.211	0.029	0.837
FastSLAM (50 parts)	1.704	0.077	4.397
FastSLAM (300 parts)	0.424	0.047	1.470
FastSLAM (500 parts)	0.175	0.046	0.628

5.3 Discussion

To visualize the behavior of the filters, we present the estimated trajectories with error over time and map quality. Figures 3 and 7 illustrate these metrics for EKF-SLAM and the best-performing FastSLAM configuration ($M = 500$), respectively on Dataset 1, while Figure 4 and 7 show the final maps obtained.

The UTIAS datasets contain a relatively small number of landmarks (15 in total), which strongly favors EKF-SLAM from a computational perspective. Since the quadratic complexity of the EKF update step remains low at this scale, EKF-SLAM consistently achieves fast execution times while maintaining strong performances across all metrics.

In terms of global estimation accuracy, FastSLAM with 500 particles achieved the best overall performance in the majority of runs, particularly in terms of ATE and final map RMSE. However, the improvement over EKF-SLAM is marginal, and the two approaches produce very similar results on both trajectory and map quality. Importantly, achieving this level of accuracy with FastSLAM required a large number of particles, which significantly increased the computational cost. This trade-off is evident when comparing the 50- and 300-particle configurations, where insufficient particles lead to diverged trajectory estimates for the 50- (Figure 5) and higher errors for 300-, while having more than 500 particles leads to execution times that we decided to not consider.

Furthermore, FastSLAM exhibits variability across runs due to its stochastic nature. While the $M = 500$ configuration often performs well, some executions resulted in worse accuracy than EKF-SLAM, despite identical experimental conditions. We observed that, on average, the ATE accuracy of FastSLAM ($M=500$) oscillated between 0.210 and 0.150 meters.

An interesting result is the notable difference in the RPE metric. FastSLAM, despite achieving better global accuracy, consistently provided a worse relative pose estimate. This behavior can be attributed to the particle-based nature of the algorithm: the sampling and resampling processes introduce local pose fluctuations, and the extraction of a single pose estimate from a set of particles further amplifies short-term noise. While these effects do not significantly impact global trajectory alignment, they degrade the accuracy of short-term relative motion estimation, resulting in higher RPE values.

Overall, the results suggest that for small-scale environments with known data association and frequent landmark observations, EKF-SLAM remains a highly effective solution, offering a favorable balance between accuracy and computational efficiency. In contrast, FastSLAM does not fully exploit its theoretical advantages in this setting. Another observation that can be made from those results is that given the better RPE, as well as the execution time, EKF-SLAM seems more suited for a real-time application compared to FastSLAM in this specific environment and settings.

We think it is important to note that these conclusions are specific to the scale and structure of the evaluated datasets. In larger environments, where the quadratic complexity of EKF-SLAM becomes prohibitive, FastSLAM—when implemented with efficient data structures such as balanced binary trees—it is proven to be able to achieve superior scalability and robustness [5, 10]. Therefore, while EKF-SLAM dominates in the presented experiments, FastSLAM remains the preferred choice for

many large-scale SLAM problems.

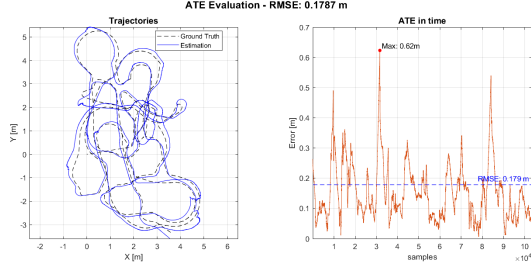


Figure 3: EKF-SLAM Absolute Trajectory Error (ATE) over time on Dataset 1 Robot 1. The black dashed line represent the groundtruth and the solid blue line the estimate

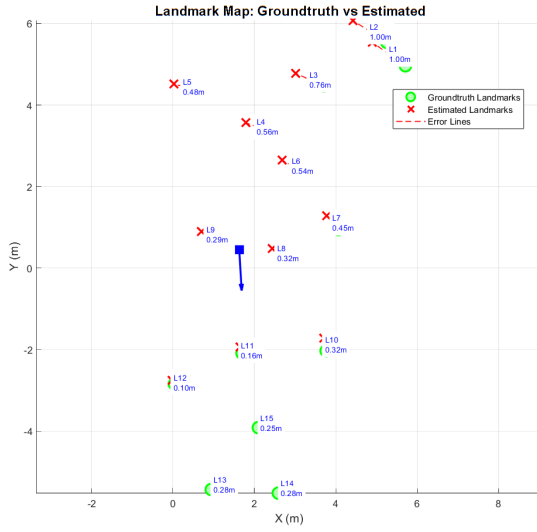


Figure 4: EKF-SLAM Final estimated map compared to ground truth on Dataset 1 Robot 1. Red Crosses represent the estimate poses and green dots the groundtruth positions

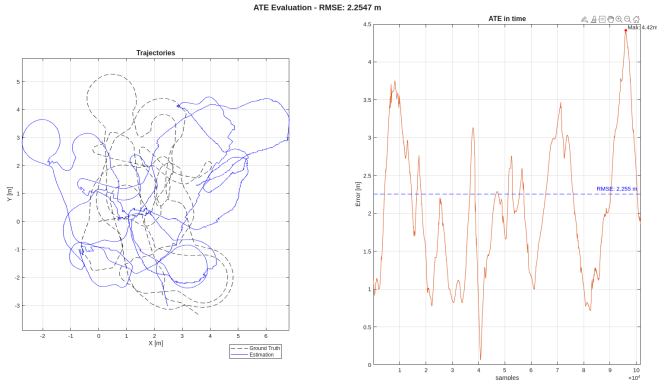


Figure 5: FastSLAM (M=50) Absolute Trajectory Error (ATE) over time on Dataset 1 Robot 1.

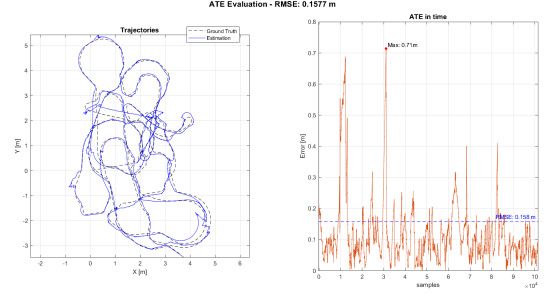


Figure 6: FastSLAM (M=500) Absolute Trajectory Error (ATE) over time on Dataset 1 Robot 1.

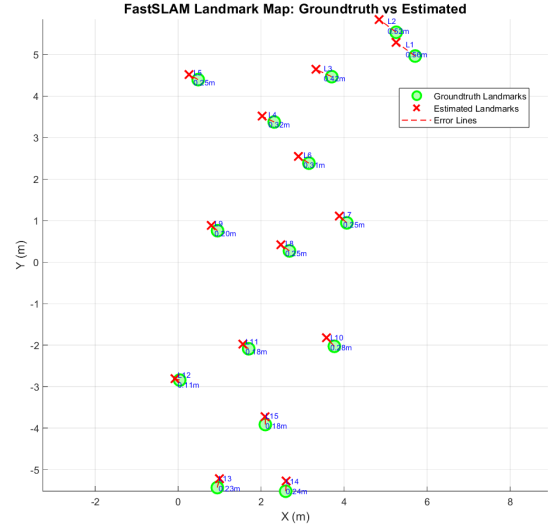


Figure 7: FastSLAM (M=500) Final estimated map compared to ground truth.

6 Conclusion

In this work, we evaluated the performance of EKF-SLAM and FastSLAM on an indoor small-scale map with known correspondences. Our results indicate that for this specific dataset, EKF-SLAM provided the best trade-off between accuracy and computational load. While the FastSLAM 1.0 algorithm—specifically the configuration with $M = 500$ particles—achieved the best absolute accuracy, the computational cost required to match the precision of the EKF was disproportionate given the sparsity of the environment. The discussion highlighted clear trade-offs between the two approaches. EKF-SLAM remains a highly valid and robust algorithm for scenarios characterized by sparse landmarks and reliable odometry, largely due to its deterministic nature and fast execution time. Consequently, it is well-suited for applications in bounded, known environments, such as an automated warehouse of medium/small dimensions. Conversely, while FastSLAM struggled to justify its computational overhead on this small map, its theoretical advantages suggest it is better suited for large-scale mappings, such as the exploration of unstructured outdoor environ-

ments, where its ability to handle high landmark counts without quadratic complexity becomes decisive.

6.1 Future Work

Future investigations should focus on validating these findings across a broader range of datasets, particularly those featuring significantly larger maps. In such scenarios, the efficient implementation of FastSLAM using balanced binary trees could be effectively evaluated to demonstrate its scalability against EKF-SLAM. Additionally, further improvements could be realized by implementing FastSLAM 2.0, which would likely improve the efficient use of particles, potentially reducing the number required to achieve convergence and mitigating the stochastic issues observed in this study.

References

- [1] N. Bergman. Posterior cramer-rao bounds for sequential estimation. In *Sequential Monte Carlo methods in practice*, pages 321–338. Springer, 2001.
- [2] B. K. Horn. Closed-form solution of absolute orientation using unit quaternions. *Journal of the optical society of America A*, 4(4):629–642, 1987.
- [3] J. J. Leonard and H. F. Durrant-Whyte. Mobile robot localization by tracking geometric beacons. *IEEE Transactions on robotics and Automation*, 7(3):376–382, 1991.
- [4] K. Y. Leung, Y. Halpern, T. D. Barfoot, and H. H. Liu. The utias multi-robot cooperative localization and mapping dataset. *The International Journal of Robotics Research*, 30(8):969–974, 2011.
- [5] M. Montemerlo, S. Thrun, D. Koller, B. Wegbreit, et al. Fastslam: A factored solution to the simultaneous localization and mapping problem. *Aaai/iaai*, 593598(2):593–598, 2002.
- [6] M. Montemerlo, S. Thrun, D. Koller, B. Wegbreit, et al. Fastslam 2.0: An improved particle filtering algorithm for simultaneous localization and mapping that provably converges. In *IJCAI*, volume 3, pages 1151–1156, 2003.
- [7] K. Petrovski, S. Jovanovski, M. Mirchev, and L. Basnarkov. On the kalman filter approach for localization of mobile robots. In *International Conference on ICT Innovations*, pages 123–133. Springer, 2016.
- [8] R. C. Smith and P. Cheeseman. On the representation and estimation of spatial uncertainty. *The international journal of Robotics Research*, 5(4):56–68, 1986.
- [9] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of rgb-d slam systems. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 573–580. IEEE, 2012.
- [10] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, 2005.